

DOKUMEN TUGAS BESAR

IF2224 Teori Bahasa Formal dan Automata

Arion Compiler : Milestone 2 - Syntax Analysis



Nama Anggota :

Zahran Alvan Putra Winarko	13524124
Neswa Eka Anggara	13524136
Daniel Putra Rywandi S.	13524143
Muh. Hartawan Haidir	13524147

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA - TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025/2026

BAB I

TEORI SINGKAT

1. Parser

Parser merupakan tahap kedua dalam proses kompilasi setelah Lexical Analysis. Parser menerima masukan berupa deretan token yang dihasilkan oleh lexer dan bertugas memeriksa apakah urutan token tersebut membentuk struktur yang valid sesuai dengan aturan grammar dari bahasa pemrograman yang dikompilasi. Selain memvalidasi urutan token, parser juga bertugas melaporkan syntax error jika ditemukan urutan yang tidak sesuai.

2. Parse Tree

Parse Tree adalah representasi struktur hierarkis dari urutan token yang sesuai dengan Context-Free Grammar. Dalam Parse Tree, seluruh struktur gramatikal direpresentasikan secara eksplisit.

- Root: Merupakan simbol non-terminal awal
- Internal Nodes : Merupakan simbol non terminal yang memiliki aturan produksi.
- Leaves : Merupakan simbol Terminal

Parse tree berbeda dengan Abstract Syntax Tree, di mana Parse Tree masih memuat elemen-elemen detail seperti kurung dan tanda baca, menjadikannya representasi lengkap dari alur penurunan grammer.

3. Recursive Descent Parser

Recursive Descent adalah salah satu metode parsing top-down di mana setiap simbol non-terminal dalam grammar diimplementasikan sebagai sebuah fungsi tersendiri. Proses parsing dimulai dari fungsi akar, yang kemudian akan memanggil fungsi-fungsi non-terminal lainnya sesuai dengan aturan produksi secara rekursif dari kiri ke kanan.

BAB II

PERANCANGAN & IMPLEMENTASI

1. Struktur Program dan Manajemen Token

Struktur program Arion Compiler dirancang secara modular. Proses *syntax analysis* dikendalikan melalui kelas Parser yang menerima masukan berupa kumpulan token (`std::vector<Token>`) yang dihasilkan dari tahapan *lexical analysis*. Manajemen dan penelusuran token dikontrol melalui sebuah kursor (`currentIndex`). Perpindahan atau navigasi dari token menggunakan dua fungsi utama, yaitu `peek()` dan `advance()`. Fungsi `peek()` bekerja sebagai *lookahead* untuk mengevaluasi tipe token saat ini tanpa memajukan kursor, sedangkan fungsi `advance()` akan mengembalikan token yang sedang ditunjuk dan kemudian memajukan kursor ke pembacaan token berikutnya.

2. Implementasi Parser

Pada program ini, Parser diimplementasikan menggunakan algoritma *Recursive Descent* dengan pendekatan berbasis *Object-Oriented Programming* menggunakan C++. Kelas Parser membaca array atau vector token yang dihasilkan dari tahap Lexer.

Secara teknis mekanismenya dibagi menjadi beberapa mekanisme utama:

- Fungsi Pengamat (Peek & Advance): Program menggunakan fungsi `peek()` untuk "mengintip" token apa yang sedang dibaca saat ini tanpa menggeser kursor pembacaan. Jika token sudah diproses dan dinilai valid, fungsi `advance()` akan dipanggil untuk menggeser kursor ke token berikutnya.
- Fungsi Pencocokan (Match): Fungsi `match(TokenType expectedType)` adalah inti pengamanan dari parser ini. Fungsi ini bertugas membandingkan token yang sedang "diintip" dengan aturan tata bahasa (*grammar*) Arion. Jika cocok, token tersebut akan dijadikan cabang terminal (daun) di dalam *Parse Tree*. Jika tidak cocok, fungsi ini akan langsung melemparkan (*throw*) pesan *Syntax Error*.
- Pemanggilan Rekursif (Recursive Descent): Setiap *non-terminal* (aturan grammar) diubah menjadi fungsi spesifik, seperti `parseProgram()`, `parseCompoundStatement()`, hingga `parseIfStatement()`. Proses berjalan dari atas ke bawah (*Top-Down*). Contohnya pada fungsi `parseProgram()`, program akan membuat "kotak" baru bernama `<program>`.

Kemudian, kotak ini diisi dengan memanggil fungsi anak-anaknya secara berurutan: `parseProgramHeader()`, lalu `parseBlock()`, dan diakhiri dengan `match(TokenType::PERIOD)`. Karena fungsi memanggil fungsi lain hingga mentok ke token terminal, proses ini disebut rekursif.

- Percabangan Logika (Branching): Pada titik-titik tertentu di mana grammar memiliki banyak pilihan (seperti pada `<statement>`), parser menggunakan percabangan if-else dengan memeriksa token saat ini (`peek().type`). Misalnya, jika token yang diintip adalah `IFSY`, maka alur akan diarahkan ke `parseIfStatement()`. Jika `WHILESY`, akan diarahkan ke `parseWhileStatement()`, dan seterusnya. Hal ini mencegah ambiguitas saat membaca struktur kode.

3. Implementasi Node

Representasi dari *parse tree* dibentuk menggunakan struktur data pohon *n-ary* melalui kelas `ParseTreeNode`. Setiap instansiasi dari *node* akan memiliki dua atribut utama, yaitu `name` (bertipe `std::string`) dan `children` (bertipe `std::vector<ParseTreeNode*>`). Atribut `name` digunakan untuk menyimpan label *grammar* atau nilai dari token, sedangkan atribut `children` digunakan untuk menyimpan referensi pointer menuju simpul-simpul turunannya. Pohon dibangun secara dinamis melalui alokasi memori (`new ParseTreeNode`) yang kemudian dirangkai menggunakan metode `addChild()`.

4. Error Handling

Penanganan kesalahan (error handling) pada parser ini diimplementasikan menggunakan mekanisme Exception Handling bawaan C++, yaitu dengan melempar (`throw`) sebuah custom exception. Jika parser menemukan urutan token yang melanggar aturan tata bahasa (*grammar*), program tidak akan langsung crash secara paksa, melainkan akan melempar pesan kesalahan yang terstruktur.

Secara teknis, implementasinya terbagi menjadi dua komponen utama:

1. Class `SyntaxError`: Program memiliki kelas khusus yang diturunkan dari `std::runtime_error`. Kelas ini bertugas merangkai pesan error yang sangat detail dan informatif. Pesan yang dihasilkan mencakup informasi baris (`line`), kolom (`col`), token apa yang sebenarnya ditemukan (`found`), dan token apa yang seharusnya ada (`expected`).

2. Pendeteksian di fungsi `match()`: Evaluasi kesalahan dilakukan di dalam fungsi `match()`. Sebelum kursor maju ke token berikutnya, fungsi ini akan mengecek apakah token yang diintip (`peek()`) sesuai dengan ekspektasi. Jika token sudah habis sebelum waktunya (EOF) atau token tidak cocok, fungsi ini langsung melempar (`throw`) objek `SyntaxError`.

Nantinya, lemparan error ini akan ditangkap (menggunakan blok `try...catch`) oleh program pemanggil utama untuk dicetak ke terminal dengan rapi, sehingga programmer pengguna bahasa Arion tahu persis di mana letak kesalahan sintaksisnya.

BAB III

PENGUJIAN

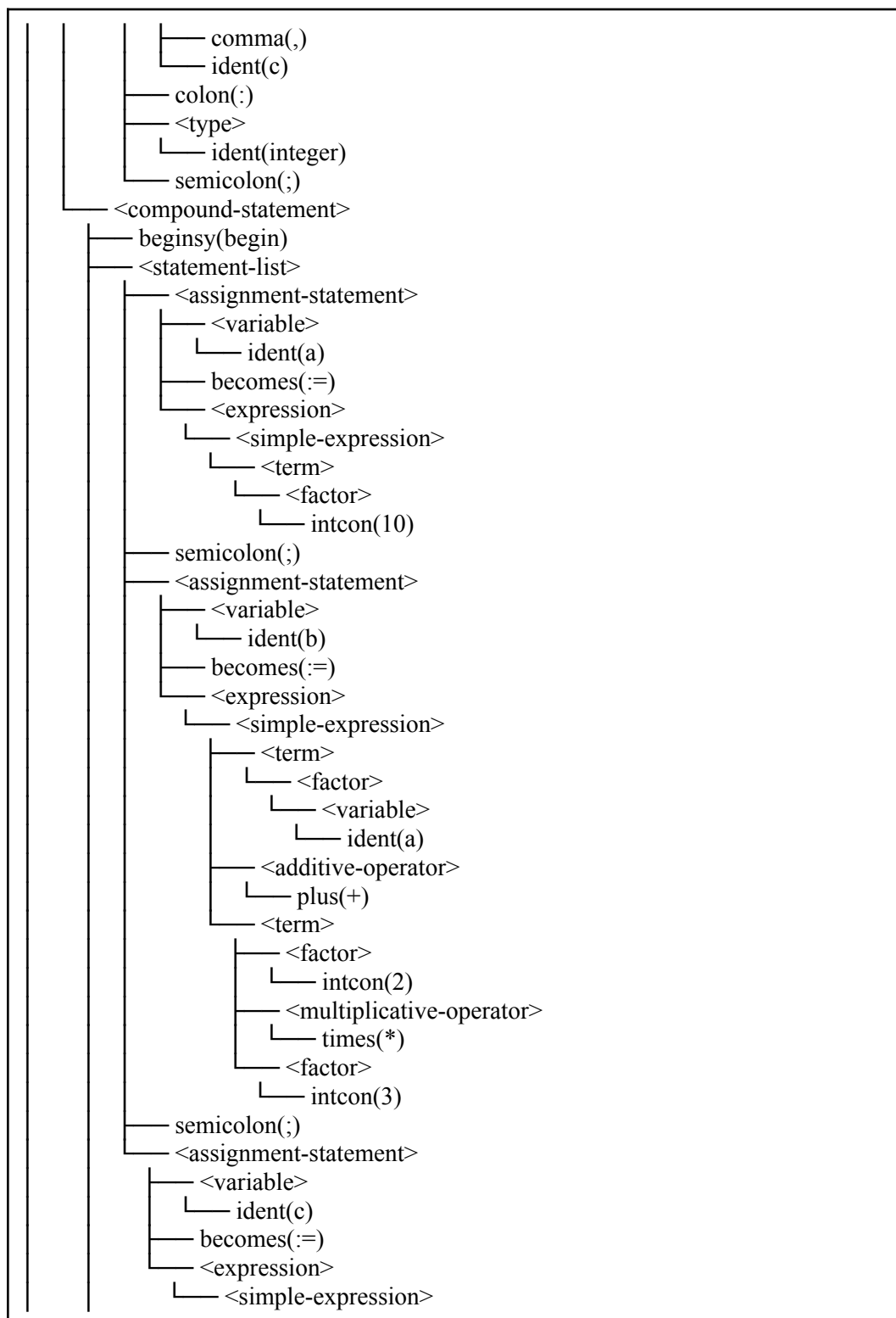
1. Test Case 1

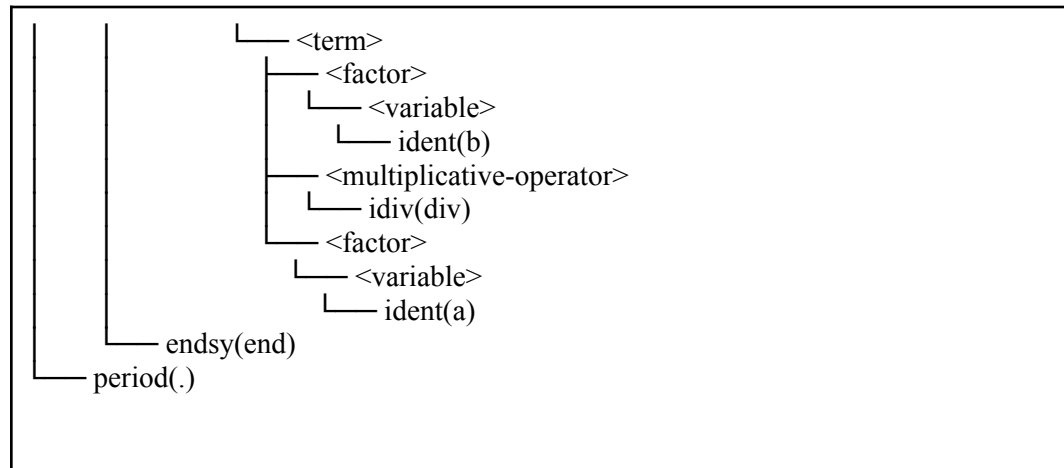
1.1. Input:

```
program Hitung;  
var a, b, c: integer;  
begin  
  a := 10;  
  b := a + 2 * 3;  
  c := b div a  
end.
```

1.2. Output:

```
<program>  
├── <program-header>  
│   ├── programsy(program)  
│   ├── ident(Hitung)  
│   └── semicolon(;  
├── <block>  
│   ├── <declaration-part>  
│   │   ├── <var-declaration>  
│   │   │   ├── varsy(var)  
│   │   │   └── <identifier-list>  
│   │   │       ├── ident(a)  
│   │   │       ├── comma(,  
│   │   │       └── ident(b)  
│   └── ...  
└── ...
```



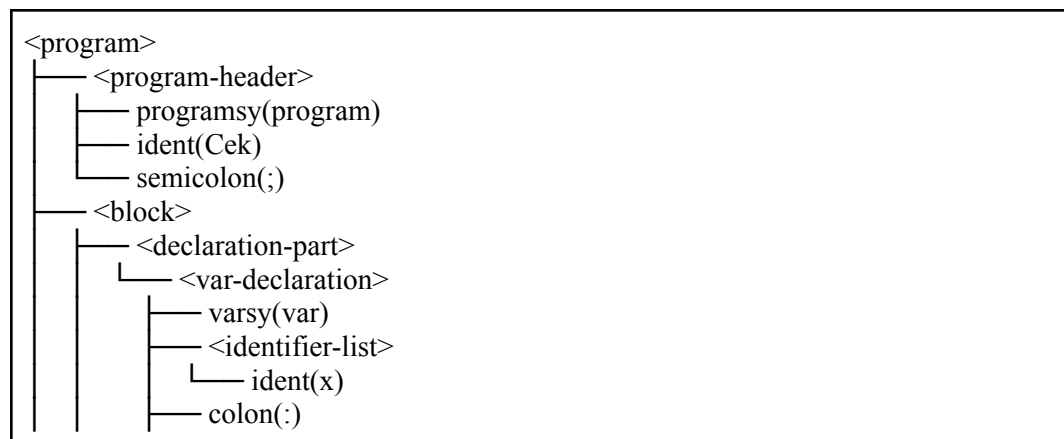


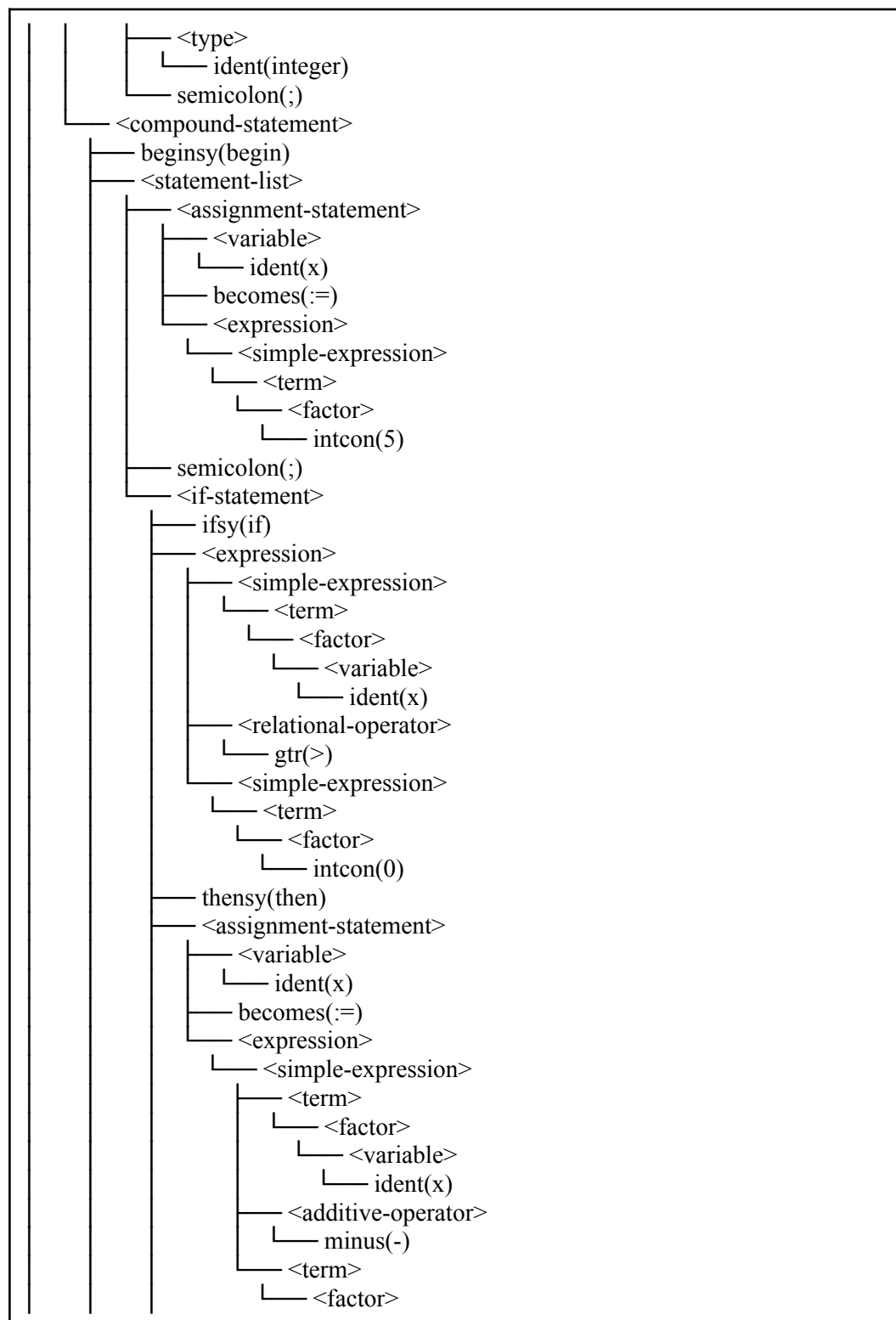
2. Test Case 2

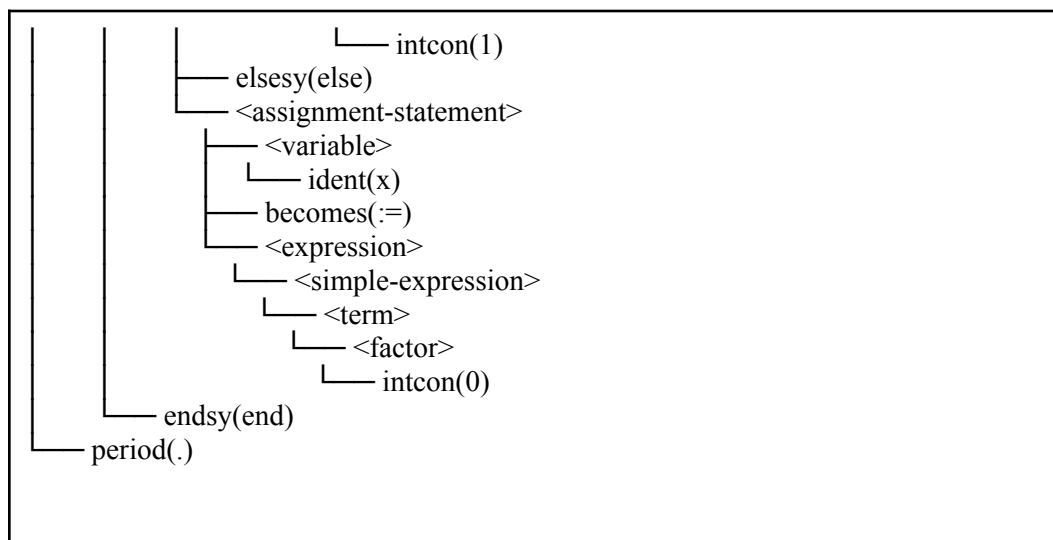
2.1. Input:

```
program Cek;
var x: integer;
begin
  x := 5;
  if x > 0 then
    x := x - 1
  else
    x := 0
end.
```

2.2. Output:







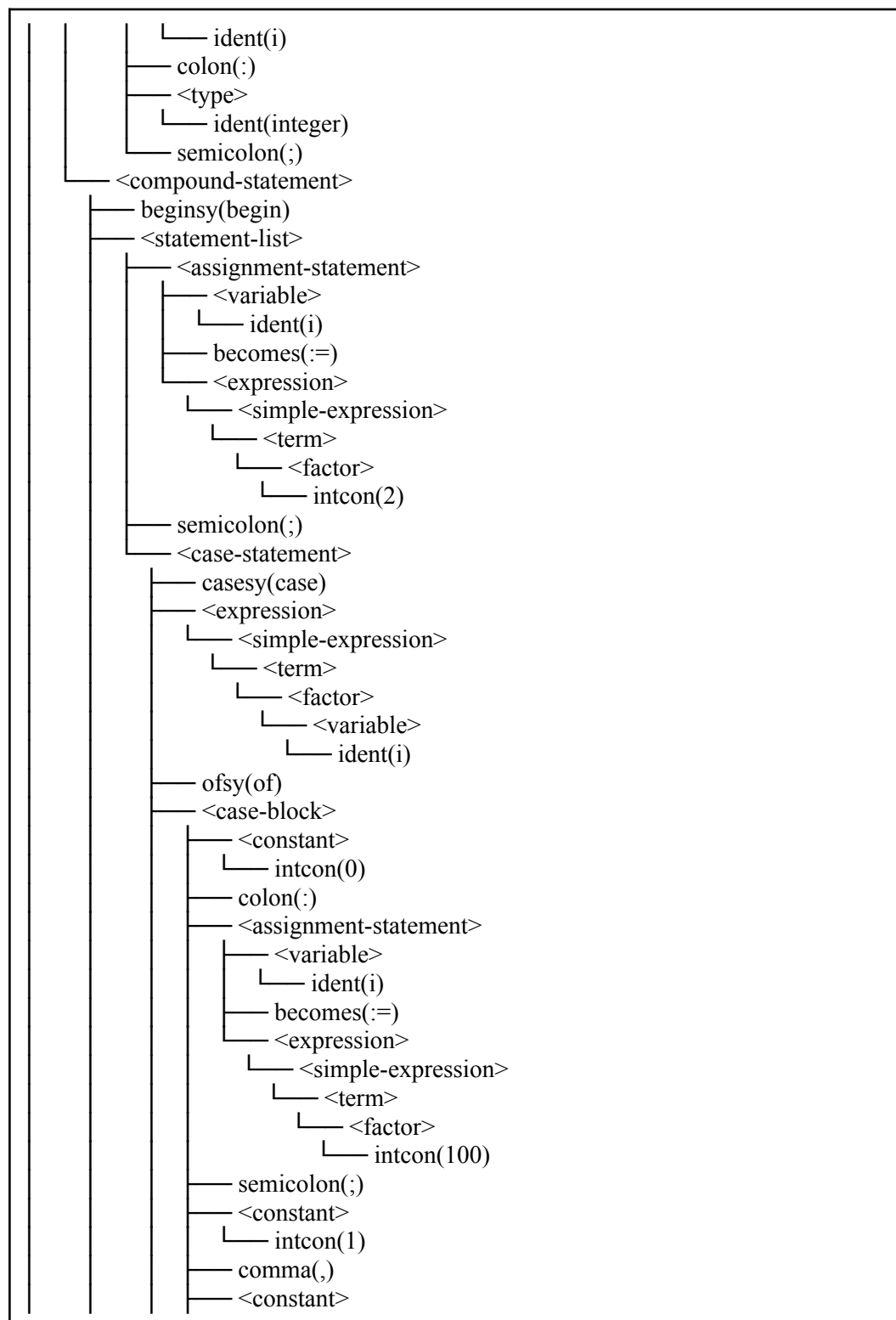
3. Test Case 3

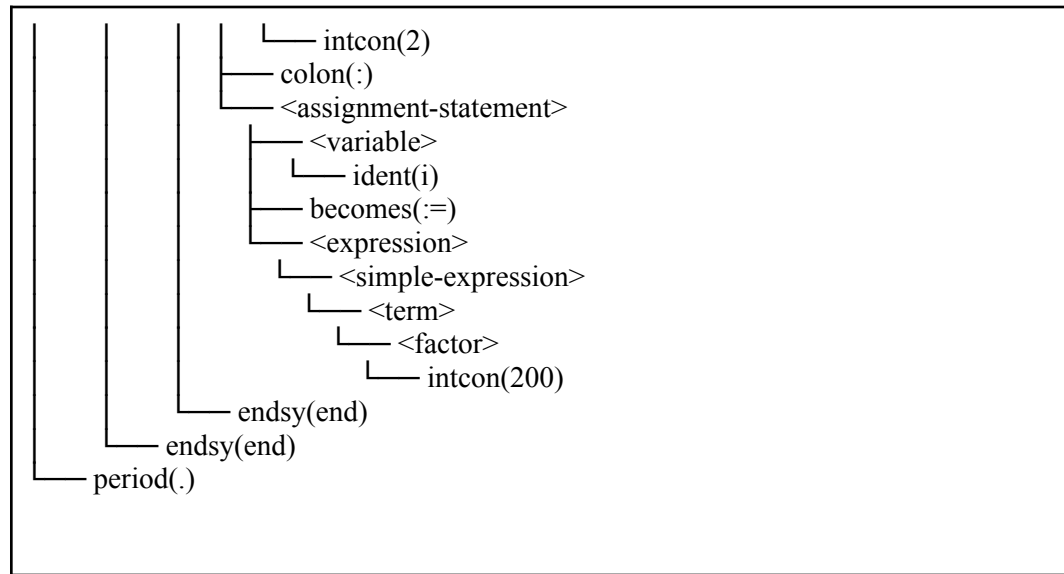
3.1. Input:

```
program Pilih;
var i: integer;
begin
  i := 2;
  case i of
    0: i := 100;
    1, 2: i := 200
  end
end.
```

3.2. Output:

```
<program>
├── <program-header>
│   ├── programsy(program)
│   ├── ident(Pilih)
│   └── semicolon(;)
└── <block>
    ├── <declaration-part>
    │   ├── <var-declaration>
    │   │   ├── varsy(var)
    │   │   └── <identifier-list>
    └──
```



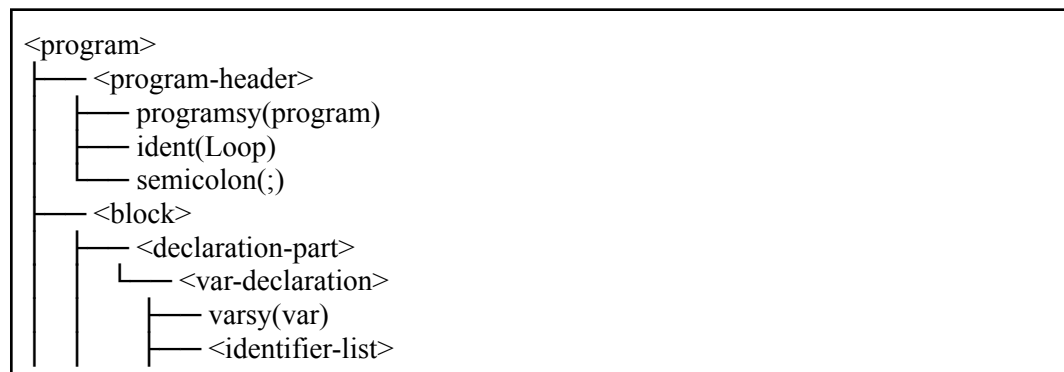


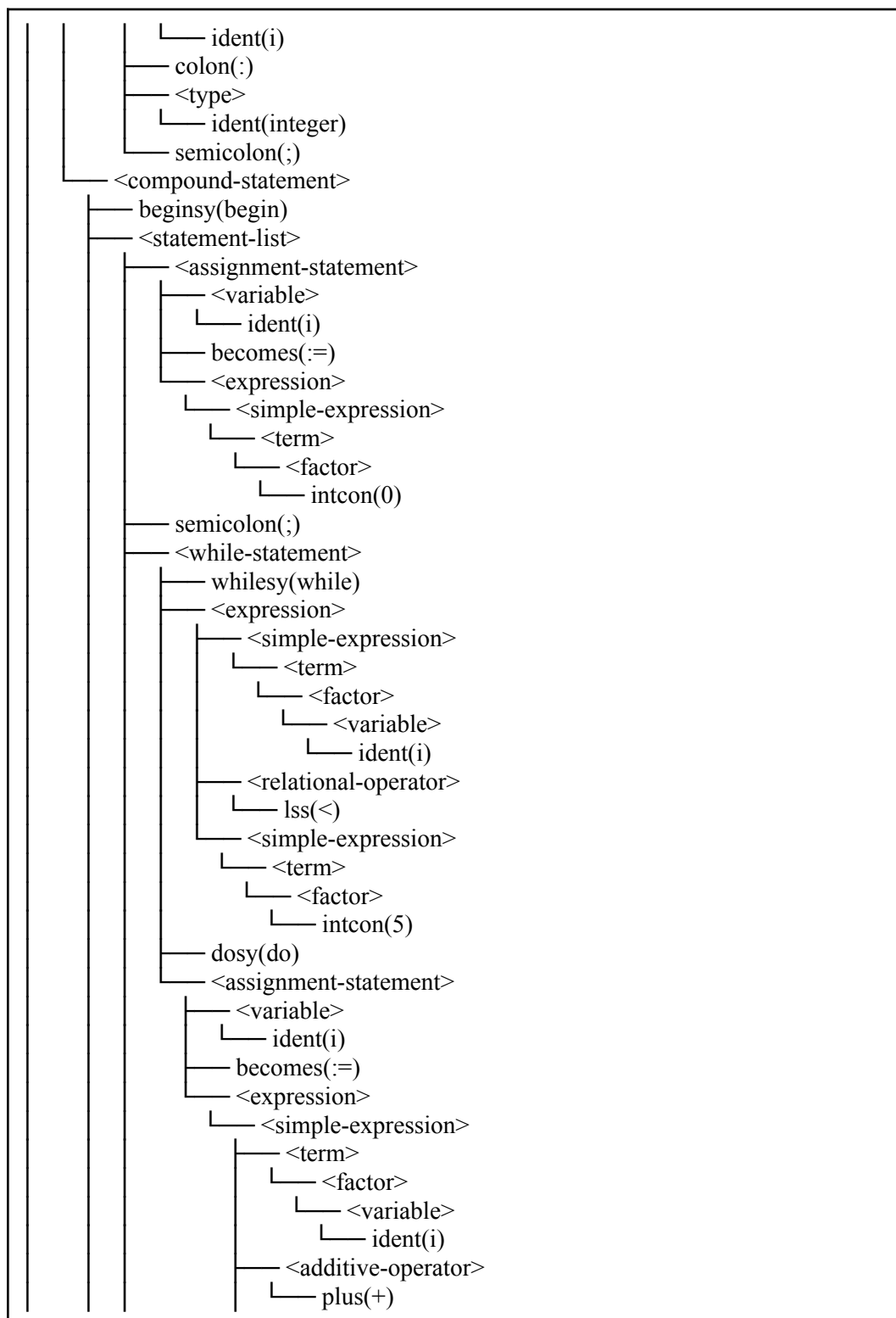
4. Test Case 4

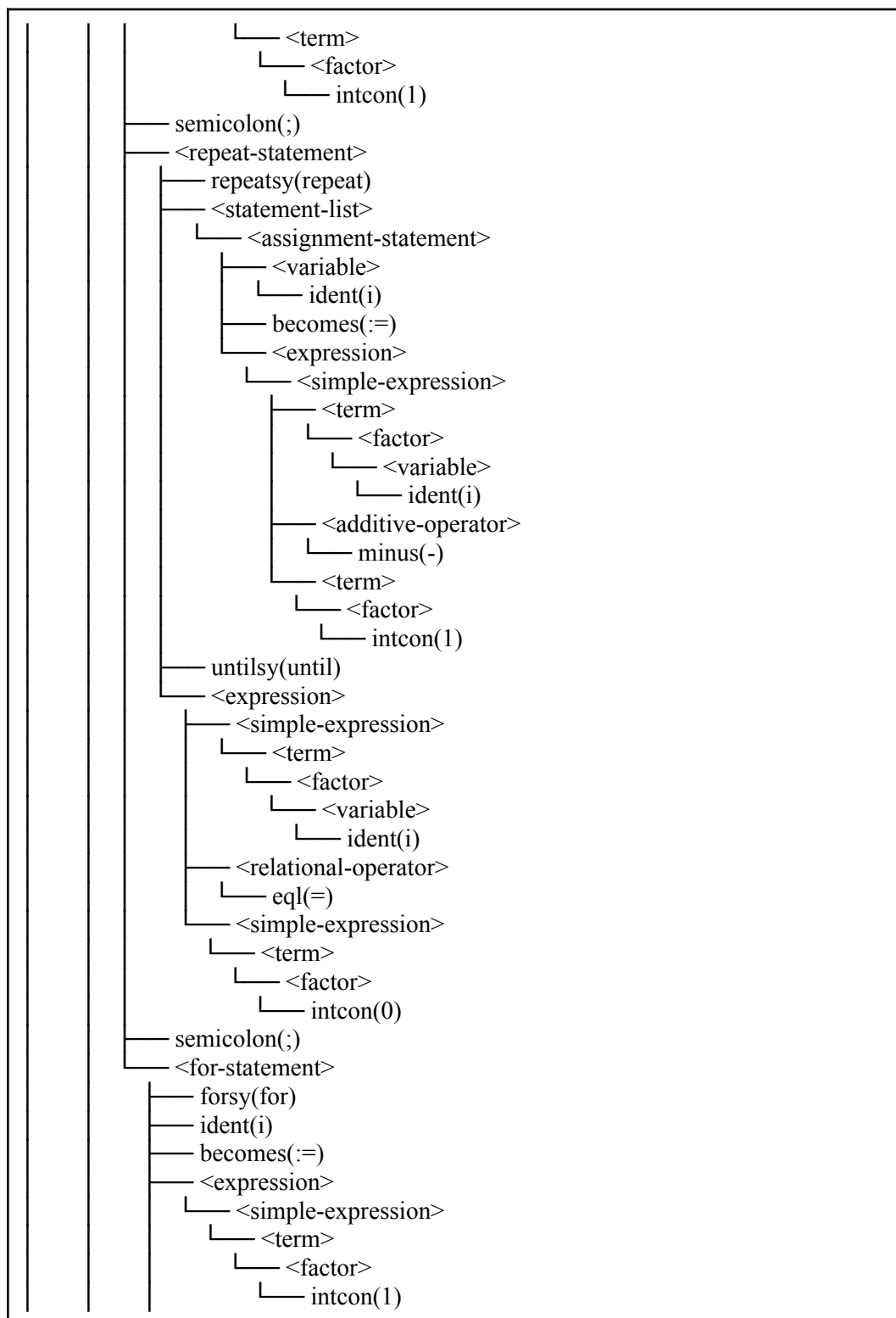
4.1. Input:

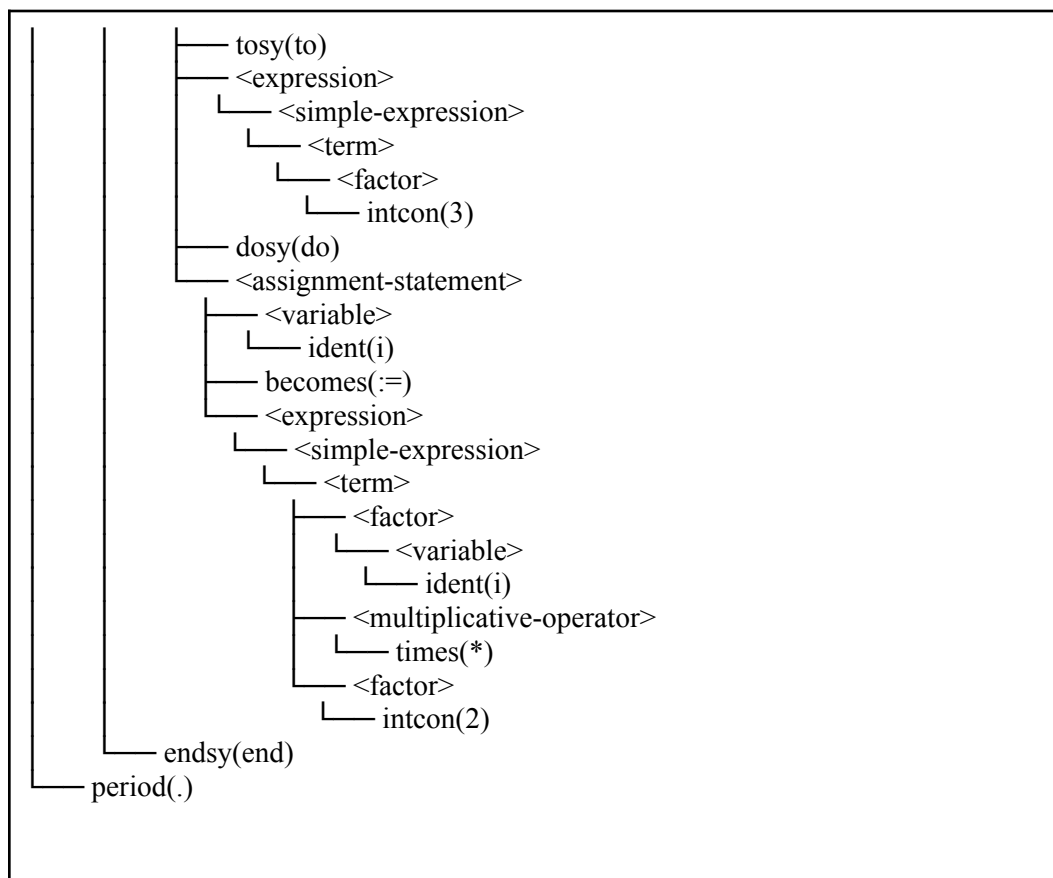
```
program Loop;
var i: integer;
begin
  i := 0;
  while i < 5 do i := i + 1;
  repeat i := i - 1 until i = 0;
  for i := 1 to 3 do i := i * 2
end.
```

4.2. Output:









BAB IV

KESIMPULAN DAN SARAN

1. Kesimpulan

Implementasi algoritma *Recursive Descent* pada proyek Milestone 2 telah berhasil mengubah deretan token dari modul *Lexer* menjadi struktur data hierarkis *Parse Tree* yang merepresentasikan *grammar* Arion. Modularitas perancangan kode di C++ dan pemanfaatan sistem *Exception* telah menghasilkan program yang menyajikan analisis sintaks (*Syntax Analysis*) dengan baik.

2. Saran

Meskipun Syntax Analyzer ini sudah berjalan dengan baik sesuai spesifikasi, terdapat beberapa ruang untuk pengembangan dan optimasi, terutama sebagai persiapan menuju tahap kompilasi berikutnya (*Semantic Analysis*):

- Transisi Menuju Abstract Syntax Tree (AST): Parse Tree yang dihasilkan saat ini masih memuat semua token secara detail, termasuk tanda baca seperti titik koma (;), tanda kurung, dan kata kunci yang sebenarnya tidak memiliki nilai komputasi. Ke depannya, pohon ini perlu disederhanakan menjadi Abstract Syntax Tree (AST), di mana node-node yang tidak penting dibuang sehingga struktur pohon menjadi lebih padat, ringkas, dan fokus pada inti operasi saja.
- Penerapan Mekanisme Error Recovery (Pemulihan Kesalahan): Saat ini, program menggunakan pendekatan fail-fast, di mana parser akan langsung melempar exception dan berhenti bekerja pada Syntax Error pertama yang ditemukan. Untuk pengembangan lebih lanjut, dapat diterapkan teknik Error Recovery (misalnya Panic Mode), di mana parser dapat mengabaikan beberapa token yang salah dan mencoba melanjutkan proses parsing. Dengan begitu, programmer bisa melihat semua kesalahan sintaks dalam satu kali run, tidak perlu memperbaikinya satu per satu.
- Manajemen Memori (Memory Management): Karena pembuatan node pada C++ saat ini menggunakan alokasi memori dinamis (raw pointers dengan keyword new), ada potensi



terjadinya memory leak jika pohon tidak dihapus dengan benar setelah digunakan. Disarankan untuk menggunakan Smart Pointers (seperti `std::unique_ptr` atau `std::shared_ptr`) dari pustaka standar C++ untuk pengelolaan memori yang lebih aman secara otomatis.

BAB V

LAMPIRAN

1. Link Repository GitHub

Berikut adalah tautan untuk mengakses repository pada platform GitHub:

<https://github.com/Alvacodee/KOI-Tubes-IF2224-2026>

2. Link Workspace Diagram Transisi DFA

Berikut adalah tautan untuk mengakses workspace pada platform draw.io:

<https://drive.google.com/file/d/1RiAAAnoB2hjkNPZm29Pn6T1FA5qK6AiLD/view?usp=sharing>

3. Pembagian Tugas

NAMA	NIM	PEMBAGIAN TUGAS	Persentase Kontribusi
Zahran Alvan Putra Winarko	13524124	- Expression + procedure call + output - expression, term, factor - parse tree + error handling	25%
Neswa Eka Anggara	13524136	- Declaration & type	25%



		- const, type, var, array, record	
Daniel Putra Rywandi S.	13524143	- Core parser + program structure - <program>, <program-header>, <block> - koordinasi flow parsing	25%
Muh. Hartawan Haidir	13524147	- Statement & control flow - if, case, while, repeat, for - statement-list, compound-statement	25%





BAB VI

REFERENSI

1. Edunex ITB. (n.d.). *Formal Language and Automata Theory Materials*. Diakses pada 7 Mei 2026, dari <https://edunex.itb.ac.id/>